Design and Implementation of a 16-Bit Instruction Set Processor

Based on the M. Morris Mano Architecture

Project Report & Engineering Documentation

Logisim Simulation Environment

1. Executive Summary & Abstract

This comprehensive engineering report details the design, construction, debugging, and verification of a fully functional 16-bit Central Processing Unit (CPU). The processor was designed from scratch at the logic-gate level within the Logisim simulation environment, adhering closely to the classical M. Morris Mano computer architecture.

The completed architecture features a common 16-bit system bus, a custom hardwired Arithmetic Logic Unit (ALU), a synchronous two-phase control unit, and a bespoke Instruction Set Architecture (ISA) capable of both memory-reference and register-reference operations. Through rigorous testing of branching logic, bitwise manipulation, and hardware 2's complement arithmetic, the processor has been verified as Turing complete and fully capable of executing complex algorithmic programs.

2. Core System Architecture

The processor operates on a von Neumann architecture, meaning that both executable instructions and raw data reside within the same unified Random Access Memory (RAM) block. The system utilizes a common bus topology to route data between physical components.

2.1 Hardware Registers

To decouple memory latency from ALU execution speed, the CPU utilizes dedicated physical registers constructed from arrays of D-type Flip-Flops. Each register serves a distinct purpose in the instruction lifecycle:

Register	Width	Description and Functionality
Program Counter (PC)	12 bits	Maintains the memory address of the *next* instruction to be fetched. It features an integrated counter to automatically increment after a fetch.
Address Register (AR)	12 bits	Acts as the sole physical gatekeeper to the RAM's address lines. Prevents bus collisions by holding the target address during execution phases.
Instruction Register (IR)	16 bits	Holds the currently executing fetched instruction. Connected to a dedicated splitter to feed the Control Unit's opcode decoders.
Accumulator (AC)	16 bits	The primary working register. All ALU operations read from and write back to the Accumulator.
Data Register (DR)	16 bits	Buffers data fetched from RAM to ensure stable voltage inputs into the ALU during timing phases.

2.2 The Common System Bus

Routing 16 physical wires between every single register would result in an unmanageable circuit. Instead, the architecture employs a Common System Bus governed by a multiplexer (MUX). During any given clock cycle, the Control Unit activates a specific 3-bit binary code to select which register is granted "write access" to the bus. All other registers are disconnected from the bus, allowing them to safely "read" the active signals without causing short circuits.

3. The Instruction Set Architecture (ISA)

The ISA defines the exact vocabulary the CPU understands. Instructions are 16 bits in length, structured identically across the memory space. The CPU identifies the type of instruction based on the highest 4 bits.

3.1 Instruction Format Breakdown

- **Bits 0–11 (12 bits):** The Operand / Address Field. Used to specify RAM locations.
- **Bits 12–14 (3 bits):** The Opcode. Provides 8 unique operations (0 through 7).
- **Bit 15 (1 bit):** The Addressing Mode Flag (*I*). `0` signifies Direct Addressing; `1` signifies Indirect Addressing.

3.2 Direct vs. Indirect Addressing (Pointers)

The processor supports memory pointers via Bit 15. In Direct Addressing, the address field points directly to the data. In Indirect Addressing, the address points to a memory slot containing *another* address, which in turn points to the data. This requires the Control Unit to execute a penalty cycle at T_3 to perform a secondary memory read, routing the nested address back into the Address Register (AR). This mechanism allows for dynamic array iteration and functional loops.

4. The Arithmetic Logic Unit (ALU)

The custom ALU is the mathematical engine of the CPU. Rather than building completely separate paths for every operation, the ALU utilizes an 8-to-1 Multiplexer controlled by the `ALU_SELECT` tunnel. The Control Unit determines the operation by manipulating these three select lines.

ALU Select Code	Operation	Assembly Command	Logical Implementation
`000`	AC AND DR	`AND`	Bitwise AND logic gates connecting AC and DR bit pairs.
`001`	AC + DR	`ADD`	Hardware Binary Adder with Carry-Out logic.
`010`	Pass DR	`LDA`	Bypasses the AC and passes the DR directly to output.
`100`	NOT AC	`CMA`	Inverts all bits of the Accumulator (1s to 0s, 0s to 1s).
`101`	Shift Right	`CIR`	Circulates bits to the right, equivalent to dividing by 2.

> **Software 2's Complement Subtraction**
>
> To optimize silicon (or logic gate) real estate, the ALU lacks a dedicated hardware subtractor. Instead, subtraction is achieved algorithmically via 2's Complement representation. To subtract B from A, the CPU loads B, inverts the bits (`CMA`), adds 1 (`INC`), and then adds A (`ADD`). This converts `A - B` into `A + (-B)` using existing hardware pathways.

5. Timing and the Control Unit

The hardwired Control Unit dictates the exact microsecond each physical component activates. It achieves this by combining decoded Opcode states (D_0 through D_7) with specific timing pulses (T_0 through T_5).

5.1 The Instruction Cycle Execution Trace

Every instruction follows a rigid rhythm. As an example, the trace below outlines the precise hardware choreography required to execute an `ADD` instruction (Opcode 1):

- **T_0 (Handshake):** The PC places the next instruction address onto the bus. The AR swallows it.
- **T_1 (Fetch):** The RAM opens the address held by the AR. The output is swallowed by the Instruction Register (IR). Simultaneously, the PC increments by 1.
- **T_2 (Decode):** The IR's opcode is decoded. The decoder lights up the D_1 wire. The address field is dropped into the AR.
- **T_3 (Idle/Indirect):** If Direct Addressing is used, the CPU idles.
- **T_4 (Data Read):** RAM opens the data address. The Data Register (DR) swallows the operand.
- **T_5 (Execute):** The Control Unit combines $D_1 \cdot T_5$ to trigger the ALU's `001` adder circuit and signals the AC to swallow the resulting sum. The Sequence Counter is cleared back to T_0.

6. Debugging and Fault Analysis

The transition from theoretical architecture to functional Logisim hardware presented several critical engineering faults. The resolution of these faults was vital to achieving system stability.

6.1 The "Floating Bus" Fault (The FFFF Bug)

Symptom: When executing the `STA` (Store) instruction, the RAM incorrectly saved `FFFF` (all 1s) regardless of what data was in the Accumulator.

Analysis: Logisim defaults disconnected or "floating" wires to an error state or all 1s. While the RAM's *Write Enable* pin was correctly receiving the $D_3 \cdot T_4$ signal, the System Bus Multiplexer was never commanded to read from the Accumulator.

Resolution: The control logic was updated to route the $D_3 \cdot T_4$ signal not only to the RAM, but also to the MUX select pins, forcing the bus to physically connect to the AC during the write cycle.

6.2 Clock Gating and the HLT / BUN Distinction

A major design challenge was preventing the CPU from accidentally fetching raw data as executable code. Two distinct hardware solutions were verified:

- `BUN` **(Branch Unconditionally):** Operates by overwriting the PC at time T_4 ($D_4 \cdot T_4$). By branching to the beginning of the program, it traps the CPU in a safe, infinite loop while power remains active.
- `HLT` **(Halt):** Achieved via physical Clock Gating. The signal $D_7 \cdot T_3 \cdot B_0$ routes to the Asynchronous Clear pin of a master Start/Stop Flip-Flop. When triggered, the flip-flop outputs a `0` into the master clock's AND gate, instantly severing electrical power to the Sequence Counter and freezing the CPU completely.

7. Master Test Suite Verification

The CPU's integrity was verified using a suite of Hexadecimal test programs loaded into RAM. The system successfully passed all operational conditions.

7.1 Arithmetic and Memory Test (Subtraction)

This program tests the ALU, Memory-Reference, and Register-Reference pathways simultaneously by performing `15 - 5 = 10` (Hex `000A`).

RAM Address	Hex Code	Assembly	Explanation
`000`	`2006`	`LDA 006`	Load Data 2 (`0005`) into the Accumulator.
`001`	`7200`	`CMA`	Flip all bits. AC becomes `FFFA`.
`002`	`7020`	`INC`	Add 1. AC becomes `FFFB` (True -5).
`003`	`1005`	`ADD 005`	Add Data 1 (`000F`) to the AC.
`004`	`7001`	`HLT`	Halt execution via clock gating.
`005`	`000F`	*Data 1*	The Minuend (15 in decimal).
`006`	`0005`	*Data 2*	The Subtrahend (5 in decimal).

Result: Verified. At the conclusion of T_5 during instruction `003`, the Accumulator held exactly `000A`.

7.2 Conditional Branching Test (SZA)

This program verifies the CPU's ability to make autonomous decisions by skipping code if the Accumulator equals zero.

RAM Address	Hex Code	Assembly	Explanation
`000`	`7800`	`CLA`	Clear AC to `0000`.
`001`	`7004`	`SZA`	Skip the next line of code if AC = `0000`.
`002`	`0000`	*Trap*	If executed, the CPU will crash/halt incorrectly.
`003`	`2005`	`LDA 005`	Load success code into AC.
`004`	`7001`	`HLT`	Halt execution.
`005`	`7777`	*Data*	Success verification data.

Result: Verified. The Program Counter successfully incremented twice during the execution of `SZA`, avoiding the trap at `002` and terminating with `7777` in the AC.

8. Conclusion

The realization of the Mano architecture within Logisim definitively proves the foundational concepts of digital logic design. By routing discrete control signals generated through precise time-state combinations, simple binary logic gates were orchestrated into a complex, decision-making machine. The resulting CPU provides a highly stable, extensible platform capable of accommodating future enhancements, such as direct Memory Access (DMA) and peripheral I/O integration.